

FPGA 与 VHDL 考试 章节变式题详解

覆盖主教材全部 23 个电路的
所有考试变式

2026 年 6 月 3 日

目录

第一部分 组合逻辑电路变式详解	6
第一章 8-3 编码器——5 大变式详解	7
1.1 变式 1: 16-4 编码器	7
1.2 变式 2: 优先级编码器 (必考!)	8
1.3 变式 3: 带使能端的编码器	10
1.4 变式 4: 带 GS (Group Select) 输出的编码器	11
1.5 变式 5: 低电平有效的编码器	11
1.6 编码器变式总结	13
第二章 3-8 译码器——5 大变式详解	14
2.1 变式 1: 74LS138 风格译码器 (低有效输出 + 三使能端) ——必考!	14
2.2 变式 2: 用译码器实现任意逻辑函数 (必考!)	15
2.3 变式 3: 用 3-8 译码器构建 4-16 译码器 (片选级联)	17
2.4 变式 4: 用译码器做地址译码	18
2.5 变式 5: 显示译码器 (BCD→7 段)	18
2.6 译码器变式总结	19
第三章 BCD 译码器——5 大变式详解	20
3.1 变式 1: 多位 BCD 动态扫描显示	20
3.2 变式 2: 带锁存的 BCD 译码器	20
3.3 变式 3: 共阳极 vs 共阴极数码管	21
3.4 变式 4: 计数器 + 译码器 + 数码管——综合题	22
3.5 变式 5: 16 进制显示译码器 (0-F)	23
第四章 4 选 1 MUX——4 大变式详解	24
4.1 变式 1: 用 MUX 实现任意逻辑函数 (必考!)	24
4.2 变式 2: 双 4 选 1 MUX 构建 8 选 1 MUX	25
4.3 变式 3: 带使能端的 MUX	25
4.4 变式 4: MUX 扩展——16 选 1	26

目录	3
第五章 4 位比较器——4 大变式详解	27
5.1 变式 1: 8 位比较器（两片 4 位级联）	27
5.2 变式 2: 只输出”相等”的比较器	28
5.3 变式 3: 比较器 +MUX 实现取最大值	28
5.4 变式 4: 带使能的比较器	28
第六章 4 位加法器——5 大变式详解	30
6.1 变式 1: 8 位/16 位加法器（级联）	30
6.2 变式 2: 减法器（用加法器实现 A-B）	30
6.3 变式 3: 加法器 + 寄存器 = 累加器	31
6.4 变式 4: 带溢出检测的加法器	31
6.5 变式 5: BCD 加法器	32
第七章 级联 MUX——4 大变式详解	33
7.1 变式 1: 用 2 选 1 MUX 构建 8 选 1 MUX	33
7.2 变式 2: 用 4 选 1 MUX 构建 16 选 1 MUX	34
7.3 变式 3: 不对称级联（构建 6 选 1 MUX）	34
7.4 变式 4: MUX 树实现逻辑函数	34
第二部分 D 触发器变式详解	35
第八章 D 触发器——核心变式详解	36
8.1 变式 1: 带异步复位的 D 触发器 vs 带同步复位的 D 触发器	36
8.2 变式 2: 带时钟使能的 D 触发器	37
8.3 变式 3: 多位 D 触发器 = 寄存器	37
8.4 变式 4: D 触发器构成 2 分频器	38
8.5 变式 5: 建立/保持时间与最大工作频率	38
第三部分 计数器变式详解	39
第九章 计数器——统一框架变式详解	40
9.1 变式 1: 带使能的模 N 计数器	40
9.2 变式 2: 带同步加载的模 N 计数器	40
9.3 变式 3: 递增/递减双向计数器	40
9.4 变式 4: 倒计时计数器（预置 $\rightarrow 0 \rightarrow$ 预置）	41
9.5 变式 5: 级联计数器——总模值 = 乘积	41
9.6 变式 6: 格雷码计数器	41

第四部分 分频器变式详解	42
第十章 分频器——全部变式详解	43
10.1 变式 1: 直接取 Q_n 实现 2^n 分频	43
10.2 变式 2: 偶数分频 (任意 M)	43
10.3 变式 3: 奇数分频 (双沿法)	43
10.4 变式 4: 占空比可调分频器	44
10.5 变式 5: 多级分频器 (分频比相乘)	44
10.6 变式 6: 秒脉冲发生器	44
第五部分 序列发生器变式详解	45
第十一章 序列发生器——全部变式详解	46
11.1 变式 1: 任意周期序列 (计数器 + 映射表)	46
11.2 变式 2: 状态机型序列发生器	46
11.3 变式 3: 移位寄存器 + LFSR 发生器	46
11.4 变式 4: ROM/查找表型序列发生器	47
第六部分 移位寄存器变式详解	48
第十二章 移位寄存器——全部变式详解	49
12.1 变式 1: 串并转换 (SIPO)	49
12.2 变式 2: 并串转换 (PISO)	49
12.3 变式 3: 双向移位	49
12.4 变式 4: 循环移位	50
12.5 变式 5: 算术移位 (保持符号位)	50
第七部分 环形计数器变式详解	51
第十三章 环形计数器——全部变式详解	52
13.1 变式 1: N 位环形计数器	52
13.2 变式 2: 约翰逊计数器 ($2N$ 个状态)	52
13.3 变式 3: 自启动环形计数器	52
13.4 变式 4: 用环形计数器实现时序控制器	53

目录	5
----	---

第八部分 序列检测器变式详解	54
----------------	----

第十四章 序列检测器——全部变式详解	55
--------------------	----

14.1 变式 1: 检测 1011 (重叠 vs 非重叠)	55
14.2 变式 2: 检测 1101	55
14.3 变式 3: 检测 1111 (连续 4 个 1)	55
14.4 变式 4: 检测 0101	56
14.5 变式 5: 用移位寄存器 + 比较器替代状态机	56

第九部分 VHDL 综合变式详解	57
------------------	----

第十五章 VHDL 综合——全部变式详解	58
----------------------	----

15.1 变式 1: if-else 的综合结果	58
15.2 变式 2: case 的综合结果	58
15.3 变式 3: signal vs variable	59
15.4 变式 4: process 的综合结果	59
15.5 变式 5: 常见 VHDL 模式 → 硬件对照表	60

第一部分

组合逻辑电路变式详解

第一章 8-3 编码器——5 大变式详解

1.1 变式 1: 16-4 编码器

完整教学

【电路升级】 8-3 编码器: $2^3 = 8$ 个输入 $\rightarrow$ 3 位输出。16-4 编码器: $2^4 = 16$ 个输入 $\rightarrow$ 4 位输出。

【设计思想】 原理完全不变——每个输出位 = 所有编号中该位为 1 的输入的 OR。
对于 16-4 编码器, 输出 $Y_3Y_2Y_1Y_0$, 输入 $I_0 \sim I_{15}$ 。

【逻辑推导】

$$Y_3 = I_8 + I_9 + I_{10} + I_{11} + I_{12} + I_{13} + I_{14} + I_{15} \quad (\text{编号 8-15 的 bit3=1}) \quad (1.1)$$

$$Y_2 = I_4 + I_5 + I_6 + I_7 + I_{12} + I_{13} + I_{14} + I_{15} \quad (\text{bit2=1 的编号}) \quad (1.2)$$

$$Y_1 = I_2 + I_3 + I_6 + I_7 + I_{10} + I_{11} + I_{14} + I_{15} \quad (1.3)$$

$$Y_0 = I_1 + I_3 + I_5 + I_7 + I_9 + I_{11} + I_{13} + I_{15} \quad (1.4)$$

【通用公式】 对于 2^n - n 编码器:

$$Y_k = \sum_{i=0}^{2^n-1} I_i \quad \text{当 } i \text{ 的第 } k \text{ 位为 1 时}$$

【VHDL】

```
1 entity encoder_16to4 is
2   port (I : in std_logic_vector(15 downto 0);
3         Y : out std_logic_vector(3 downto 0));
4 end encoder_16to4;
5
6 architecture behavioral of encoder_16to4 is
7 begin
8   process(I)
9   begin
10    case I is
11      when "0000000000000001" => Y <= "0000";
```


(x= 任意值, 不管它是 0 还是 1)

【VHDL——if-else 级联】

```

1  entity priority_encoder is
2      port (I : in std_logic_vector(7 downto 0);
3            Y : out std_logic_vector(2 downto 0);
4            GS : out std_logic);  -- Group Select: 有输入=1
5  end priority_encoder;
6
7  architecture behavioral of priority_encoder is
8  begin
9      process(I)
10     begin
11         if I(7) = '1' then
12             Y <= "111"; GS <= '1';
13         elsif I(6) = '1' then
14             Y <= "110"; GS <= '1';
15         elsif I(5) = '1' then
16             Y <= "101"; GS <= '1';
17         elsif I(4) = '1' then
18             Y <= "100"; GS <= '1';
19         elsif I(3) = '1' then
20             Y <= "011"; GS <= '1';
21         elsif I(2) = '1' then
22             Y <= "010"; GS <= '1';
23         elsif I(1) = '1' then
24             Y <= "001"; GS <= '1';
25         elsif I(0) = '1' then
26             Y <= "000"; GS <= '1';
27         else
28             Y <= "000"; GS <= '0';  -- 无有效输入
29         end if;
30     end process;
31 end behavioral;

```

【为什么 if-else 实现优先级】 if-else 的语义: 从上到下依次判断, 第一个为真的分支被执行。所以 I7 在第一个 if → I7 优先级最高。

【综合成什么硬件】 级联 MUX 链 (优先级 MUX 树)。I7→I6→...→I0 逐级选择。

【GS 信号的必要性】 没有 GS 时, 输入全 0 (无有效输入) 和只有 I0=1 时, 输出都是 000——无法区分! GS=1 表示“有有效输入”。

【Testbench——测试优先级】

```

1  -- 同时置 I7 和 I0=1 → 应该输出 111 (I7 优先)
2  I <= "10000001"; wait for 10 ns;

```

```
3 -- 输出应为111, GS=1 (I7优先级高于I0)
```

考试聚焦

必考！ 优先级编码器是考试最高频的组合逻辑变式之一。

- if-else 级联 = 优先级递减的硬件
- GS 信号区分”无输入”和”选择 I0”
- 优先级从高到低排列 if 条件

1.3 变式 3：带使能端的编码器

完整教学

【电路】 在标准编码器基础上增加 Enable(E) 输入。E=1 时编码器正常工作；E=0 时所有输出为 0（或高阻态）。

【VHDL】

```
1 entity encoder_with_enable is
2     port (I : in std_logic_vector(7 downto 0);
3           E : in std_logic;
4           Y : out std_logic_vector(2 downto 0));
5 end encoder_with_enable;
6
7 architecture behavioral of encoder_with_enable is
8 begin
9     process(I, E)
10    begin
11        if E = '0' then
12            Y <= "000"; -- 禁止输出
13        else
14            -- 正常编码逻辑
15            case I is
16                when "00000001" => Y <= "000";
17                when "00000010" => Y <= "001";
18                -- ...
```

```
19      when others => Y <= "000";
20      end case;
21      end if;
22      end process;
23  end behavioral;
```

【设计思想】使能端是数字电路中常见的控制信号。在不使用时关断输出以节省功耗/避免总线冲突。

1.4 变式 4：带 GS（Group Select）输出的编码器

完整教学

【问题】普通 8-3 编码器：当所有输入为 0 时，输出 000。但当 $I_0 = 1$ 时，输出也是 000。如何区分这两种情况？

【方案】增加 GS（Group Select/Valid）输出：

- 任意输入 =1 $\rightarrow$ GS=1（表示输出有效）
- 所有输入 =0 $\rightarrow$ GS=0（表示输出无效/无输入选中）

【VHDL】

```
1 GS <= '0' when I = "00000000" else '1';
2 -- 或
3 GS <= I(0) or I(1) or I(2) or I(3) or I(4) or I(5) or I(6) or I(7);
```

【GS 硬件】一个 8 输入 OR 门——需要 8 个输入引脚。

【考试常考】GS 信号的意义：解决编码器输出的二义性问题。

1.5 变式 5：低电平有效的编码器

完整教学

【电路变化】输入和输出都是低电平有效（0 表示有效/选中，1 表示无效）。这在实际 IC（如 74LS148）中很常见——低电平有效更节省功耗，且与 TTL 电平兼容。

【逻辑推导】低电平有效时：

- 输入端: $\overline{I_0}, \overline{I_1}, \dots, \overline{I_7}$ (上划线表示低电平有效)
- 当 $\overline{I_k} = 0$ 且其他为 1 时 $\rightarrow$ 输出编码为 k
- 用 NAND 门替代 OR 门来实现 (因为低有效的或 = 高有效的与非)

【VHDL】

```

1  entity encoder_active_low is
2      port (
3          I_n : in  std_logic_vector(7 downto 0); -- 低有效
4          Y_n : out std_logic_vector(2 downto 0); -- 低有效
5          GS_n: out std_logic -- 低有效: 0=有输入, 1=无输入
6      );
7  end encoder_active_low;
8
9  architecture behavioral of encoder_active_low is
10 begin
11     process(I_n)
12     begin
13         GS_n <= '1'; -- 默认: 无有效输入
14         Y_n <= "111";
15
16         if I_n(7) = '0' then
17             Y_n <= "000"; GS_n <= '0'; -- 7的编码=111, 反相=000
18         elsif I_n(6) = '0' then
19             Y_n <= "001"; GS_n <= '0'; -- 6=110, 反相=001
20         elsif I_n(5) = '0' then
21             Y_n <= "010"; GS_n <= '0';
22         elsif I_n(4) = '0' then
23             Y_n <= "011"; GS_n <= '0';
24         elsif I_n(3) = '0' then
25             Y_n <= "100"; GS_n <= '0';
26         elsif I_n(2) = '0' then
27             Y_n <= "101"; GS_n <= '0';
28         elsif I_n(1) = '0' then
29             Y_n <= "110"; GS_n <= '0';
30         elsif I_n(0) = '0' then
31             Y_n <= "111"; GS_n <= '0'; -- 0的编码=000, 反相=111
32         end if;
33     end process;
34 end behavioral;

```

【关键理解】低有效编码: 输入 0 有效 $\rightarrow$ 输出也是 0=111(即 7 的二进制反码)。高有效和低有效编码器的输出值互为反码。

【考试提示】看清题目要求：高有效还是低有效？74LS148 是低有效优先级编码器，这是常考器件。

1.6 编码器变式总结

编码器 5 大变式速查

变式	核心改动	VHDL 关键
16-4 编码	规模扩大	case 16 分支/for 循环
优先级编码	允许同时多输入	if-else 级联 (高 → 低)
带使能	增加 E 控制	if E='0' then Y<="000"
带 GS 输出	增加有效标志	GS<=not(I= 全 0)
低电平有效	0 有效 1 无效	判断 I(n)='0', 输出取反

第二章 3-8 译码器——5 大变式详解

2.1 变式 1: 74LS138 风格译码器（低有效输出 + 三使能端） ——必考！

完整教学

【电路】74LS138 是考试中最常出现的译码器芯片。特点：

- 3 位地址输入 A2,A1,A0
- 8 个输出 Y0-Y7，全部低电平有效（选中 =0，未选中 =1）
- 3 个使能端：G1(高有效), G2A(低有效), G2B(低有效)
- 只有三个使能端同时有效（G1=1, G2A=0, G2B=0）时，译码器才工作

【使能逻辑】 $enable = G1 \cdot \overline{G2A} \cdot \overline{G2B}$

【VHDL】

```
1 entity decoder_74ls138 is
2   port (
3     A      : in  std_logic_vector(2 downto 0);
4     G1     : in  std_logic;
5     G2A_n : in  std_logic; -- 低有效
6     G2B_n : in  std_logic; -- 低有效
7     Y_n    : out std_logic_vector(7 downto 0) -- 低有效
8   );
9 end decoder_74ls138;
10
11 architecture behavioral of decoder_74ls138 is
12   signal enable : std_logic;
13 begin
14   enable <= G1 and (not G2A_n) and (not G2B_n);
15
```

```

16 process(A, enable)
17 begin
18     if enable = '0' then
19         Y_n <= "11111111"; -- 全高=全无效
20     else
21         case A is
22             when "000" => Y_n <= "11111110"; -- Y0=0
23             when "001" => Y_n <= "11111101"; -- Y1=0
24             when "010" => Y_n <= "11111011";
25             when "011" => Y_n <= "11110111";
26             when "100" => Y_n <= "11101111";
27             when "101" => Y_n <= "11011111";
28             when "110" => Y_n <= "10111111";
29             when "111" => Y_n <= "01111111";
30             when others => Y_n <= "11111111";
31         end case;
32     end if;
33 end process;
34 end behavioral;

```

【为什么用 3 个使能端】多个使能端方便级联。G2A 和 G2B 是低有效，G1 是高有效——这种混合设计使得可以用不同信号控制，增加灵活性。

【常考陷阱】

- 看清输出是低有效还是高有效——74LS138 输出 0 表示选中
- 三个使能端必须同时满足条件译码器才工作
- 使能无效时所有输出为 1（全高 = 全无效）

2.2 变式 2：用译码器实现任意逻辑函数（必考！）

完整教学

【核心原理】译码器的每个输出 = 一个最小项。任何组合逻辑函数 = 若干最小项之和（OR）。

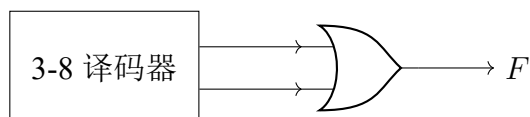
【通用公式】 $F(A, B, C) = \sum m(\dots) = Y_{i1} + Y_{i2} + Y_{i3} + \dots$

【例题】用 3-8 译码器 + 一个 OR 门实现 $F(A, B, C) = \sum m(1, 3, 5, 7)$ 。

【解法】

1. 译码器产生 8 个最小项: $Y_0=m_0, Y_1=m_1, \dots, Y_7=m_7$
2. $F = m_1 + m_3 + m_5 + m_7 = Y_1 + Y_3 + Y_5 + Y_7$
3. 用 1 个 4 输入 OR 门连接 Y_1, Y_3, Y_5, Y_7

【电路结构】



$A_2 A_1 A_0$ = 变量 A,B,C, OR 门输入 = Y_1, Y_3, Y_5, Y_7

【VHDL】

```

1  -- 例化译码器
2  dec: decoder_3to8 port map(A=>ABC, Y=>Y);
3  -- OR 门
4  F <= Y(1) or Y(3) or Y(5) or Y(7);

```

【推广：实现任意 3 变量函数】

函数	连接
$F = m_0 + m_2 + m_4 + m_6$	$Y_0 + Y_2 + Y_4 + Y_6$
$F = m_0 + m_1 + m_2 + m_3$	$Y_0 + Y_1 + Y_2 + Y_3$
$F = A$ (即 $m_4 + m_5 + m_6 + m_7$)	$Y_4 + Y_5 + Y_6 + Y_7$
$F = \overline{A}$	$Y_0 + Y_1 + Y_2 + Y_3$

【如果函数用 NAND 实现】用 NAND 门替代 OR 门（德摩根定理）。此时输出是反相的，但功能等价。

考试聚焦

必考！用 3-8 译码器 + 门电路实现任意 3 变量逻辑函数。步骤：(1) 将函数写成最小项之和 (2) 对应 Y 输出用 OR 门连接

2.3 变式 3：用 3-8 译码器构建 4-16 译码器（片选级联）

完整教学

【问题】 只有 3-8 译码器，如何得到 4 位地址 → 16 个输出的 4-16 译码器？

【方案】 两片 3-8 译码器 + 最高位地址 A3 做片选。

【级联逻辑】

- 片 0（低位）：处理 A3=0 时，地址 0000-0111（译码器输出 Y0-Y7 对应系统 Y0-Y7）
- 片 1（高位）：处理 A3=1 时，地址 1000-1111（译码器输出 Y0-Y7 对应系统 Y8-Y15）
- A3=0 → 片 0 使能，片 1 禁止
- A3=1 → 片 1 使能，片 0 禁止

【VHDL】

```
1 entity decoder_4to16 is
2     port (A : in std_logic_vector(3 downto 0);
3           Y : out std_logic_vector(15 downto 0));
4 end decoder_4to16;
5
6 architecture structural of decoder_4to16 is
7     signal Y_low, Y_high : std_logic_vector(7 downto 0);
8     component decoder_3to8 is
9         port (A : in std_logic_vector(2 downto 0);
10              E : in std_logic; Y : out std_logic_vector(7 downto 0));
11     end component;
12 begin
13     -- 片0: A3=0时使能
14     dec_low: decoder_3to8 port map(
15         A => A(2 downto 0), E => not A(3), Y => Y_low);
16
17     -- 片1: A3=1时使能
18     dec_high: decoder_3to8 port map(
19         A => A(2 downto 0), E => A(3), Y => Y_high);
20
21     Y <= Y_high & Y_low; -- 拼接为16位输出
22 end structural;
```

【通用级联公式】用 2^k 个 n - 2^n 译码器构建 $(n+k)$ - 2^{n+k} 译码器。高 k 位地址 → 片选，低 n 位地址 → 各片输入。

【常见考题】用两片 74LS138 构建 4-16 译码器。利用使能端做片选。

2.4 变式 4：用译码器做地址译码

完整教学

【应用场景】CPU 有 16 位地址总线，但连接了多个外设（RAM、ROM、IO 端口等）。用译码器将地址空间划分为不同区域，产生各外设的片选信号。

【例题】用 3-8 译码器产生 8 个外设的片选信号。地址 A15-A13 做译码输入。

【电路】

A15A14A13	选中输出	外设
000	Y0	RAM1(0x0000-0x1FFF)
001	Y1	RAM2(0x2000-0x3FFF)
010	Y2	ROM (0x4000-0x5FFF)
011	Y3	IO1 (0x6000-0x7FFF)
100	Y4	IO2 (0x8000-0x9FFF)

【核心思想】高位地址 → 译码 → 片选。低位地址 → 片内寻址。

2.5 变式 5：显示译码器（BCD→7 段）

完整教学

【问题】3-8 译码器每次只选中 1 个输出。但 7 段数码管需要同时点亮多个段（如数字“8”需要 7 段全亮）。

【区别】

- 3-8 译码器： n 输入 → 2^n 输出，输出是互斥的（恰好一个为 1）
- BCD-7 段译码器：4 输入 → 7 输出，输出不互斥（可以同时多个为 1）

【详见第三章 BCD 译码器】BCD-7 段译码器本质上是 7 个独立的组合逻辑函数，不是简单的“最小项展开”。

考试聚焦

关键区别：

- 3-8 译码器：1-of-N 选择（互斥输出）→ 用 AND 门
- BCD-7 段：任意多段同时亮（非互斥）→ 用 ROM/查找表

不搞混这两种”译码器”是考试的基本要求。

2.6 译码器变式总结

译码器 5 大变式速查

变式	核心考点	常考题型
74LS138 风格 译码器 +OR $\approx$ 任意函数 级联构建更大译码器 地址译码 vs BCD-7 段	低有效 +3 使能端 最小项之和 高位地址做片选 地址空间划分 互斥 vs 非互斥输出	写 VHDL, 判断使能条件 给定函数 → 画电路 74LS138 $\times 2 \rightarrow 4-16$ 存储器片选设计 概念辨析

第三章 BCD 译码器——5 大变式详解

3.1 变式 1：多位 BCD 动态扫描显示

完整教学

【问题】4 位数码管需要 4 个 BCD-7 段译码器吗？如果每个译码器单独驱动 → 需要 $4 \times 7 = 28$ 个 IO 口。

【方案】动态扫描：1 个译码器驱动所有数码管的段，快速轮流点亮各数码管 ($>50\text{Hz}$)，利用人眼视觉暂留。

【电路】计数器 → 2-4 译码器 → 轮流选通 4 个数码管的共阴/共阳极。BCD 译码器输出段码到所有数码管。

【VHDL 核心逻辑】

```
1 process(clk)  -- 扫描时钟(通常>200Hz)
2 begin
3     if rising_edge(clk) then
4         scan_cnt <= scan_cnt + 1;
5         case scan_cnt is
6             when "00" => digit_sel <= "0001"; seg_data <= digit0;
7             when "01" => digit_sel <= "0010"; seg_data <= digit1;
8             when "10" => digit_sel <= "0100"; seg_data <= digit2;
9             when "11" => digit_sel <= "1000"; seg_data <= digit3;
10        end case;
11    end if;
12 end process;
```

【常见考题】设计 4 位动态扫描显示电路。

3.2 变式 2：带锁存的 BCD 译码器

完整教学

【问题】BCD 译码器输入随时钟快速变化 → 数码管闪烁。需要“定格”显示。

【方案】在译码器前加寄存器。LE=1 时锁存当前 BCD 值，LE=0 时保持上次锁存值不变。

【VHDL】

```

1 process(clk)
2 begin
3     if rising_edge(clk) then
4         if LE = '1' then
5             latched_bcd <= bcd_input;  -- 锁存
6         end if;
7     end if;
8 end process;
9 seg <= bcd_to_7seg(latched_bcd);  -- 译码锁存后的值

```

【设计思想】锁存 = 寄存器 = 1 个时钟沿保存数据。本质 = 在数据通路上插入一级寄存器。

3.3 变式 3: 共阳极 vs 共阴极数码管

完整教学

【共阳极】所有 LED 阳极接 VCC。段驱动需低电平 (0) 点亮 LED。seg='0'= 亮。**【共阴极】**所有 LED 阴极接 GND。段驱动需高电平 (1) 点亮 LED。seg='1'= 亮。

【输出值关系】 $seg_{\text{共阳}} = \text{not } seg_{\text{共阴}}$ (互为反码)

【VHDL 适配】

```

1 -- 共阴极 (1 亮)
2 when "0000" => seg <= "1111110";  -- 0
3 -- 共阳极 (0 亮)
4 when "0000" => seg <= "0000001";  -- 0(反码)

```

【考试陷阱】题目未明确说明共阴/共阳时，看段码值判断：1111110(共阴 0) vs 0000001(共阳 0)。

3.4 变式 4：计数器 + 译码器 + 数码管——综合题

完整教学

【经典考题】设计 0-9 计数器 +BCD-7 段译码器 + 数码管显示。

【完整 VHDL】

```
1 entity counter_display is
2     port (clk, rst_n : in std_logic;
3           seg : out std_logic_vector(6 downto 0));
4 end counter_display;
5
6 architecture rtl of counter_display is
7     signal cnt : std_logic_vector(3 downto 0);
8     function bcd7seg(bcd : std_logic_vector(3 downto 0))
9         return std_logic_vector is
10    begin
11        case bcd is
12            when "0000" => return "1111110";
13            when "0001" => return "0110000";
14            when "0010" => return "1101101";
15            when "0011" => return "1111001";
16            when "0100" => return "0110011";
17            when "0101" => return "1011011";
18            when "0110" => return "1011111";
19            when "0111" => return "1110000";
20            when "1000" => return "1111111";
21            when "1001" => return "1111011";
22            when others => return "0000000";
23        end case;
24    end function;
25 begin
26     process(clk, rst_n)
27     begin
28         if rst_n='0' then cnt <= "0000";
29         elsif rising_edge(clk) then
30             if cnt = "1001" then cnt <= "0000";
31             else cnt <= cnt + 1;
32             end if;
33         end if;
34     end process;
35     seg <= bcd7seg(cnt);
36 end rtl;
```

【考点】两个模块整合：计数器 + 译码器。

3.5 变式 5: 16 进制显示译码器 (0-F)

完整教学

【问题】BCD 译码器只显示 0-9。如何显示 0-F (全 4 位二进制)?

【方案】扩展 case 覆盖 1010-1111, 定义 A-F 的段码。

```
1  -- 扩展BCD译码器支持0-F
2  when "1010" => return "1110111";  -- A
3  when "1011" => return "0011111";  -- b
4  when "1100" => return "1001110";  -- C
5  when "1101" => return "0111101";  -- d
6  when "1110" => return "1001111";  -- E
7  when "1111" => return "1000111";  -- F
```

【考试】有时问 A-F 的段码值, 需要记住典型表示法。

第四章 4 选 1 MUX——4 大变式详解

4.1 变式 1：用 MUX 实现任意逻辑函数（必考！）

完整教学

【核心原理】MUX 的通用表达式： $Y = \overline{S_1} \cdot \overline{S_0} \cdot D_0 + \overline{S_1} \cdot S_0 \cdot D_1 + S_1 \cdot \overline{S_0} \cdot D_2 + S_1 \cdot S_0 \cdot D_3$

如果把 S_1, S_0 接函数变量， D_k 接“该最小项是否参与=1”： D_k 接 1 → 该最小项参与函数。 D_k 接 0 → 不参与。

【结论】任何 n 变量逻辑函数 = 一个 2^n 选 1 MUX

【例题 1】用 8 选 1 MUX 实现 $F(A, B, C) = \sum m(0, 3, 5, 6)$

解： $D_0 = 1, D_1 = 0, D_2 = 0, D_3 = 1, D_4 = 0, D_5 = 1, D_6 = 1, D_7 = 0$

将 m_0 - m_7 对应的 D 分别接 1 或 0。选择线 S_2, S_1, S_0 接 A, B, C 。

【例题 2：降维法】用 4 选 1 MUX 实现 3 变量函数 $F(A, B, C) = \sum m(0, 1, 4, 6, 7)$
 $2^{3-1} = 4$ 选 1 MUX, 变量 A, B 接 S_1, S_0 , C 用于产生 D_k 。

AB	m(C=0)	m(C=1)	D 输入	逻辑
00	m0=1	m1=1	D0	1(与 C 无关)
01	m2=0	m3=0	D1	0
10	m4=1	m5=0	D2	$\overline{C}$
11	m6=1	m7=1	D3	1

$D_0=1, D_1=0, D_2=\overline{C}, D_3=1$.

【VHDL】

```
1 entity mux_logic is
2   port (A, B, C : in std_logic; F : out std_logic);
3 end mux_logic;
4 architecture rtl of mux_logic is
5   signal D : std_logic_vector(3 downto 0);
6   signal S : std_logic_vector(1 downto 0);
7 begin
8   S <= A & B;
9   D <= '1' & (not C) & '0' & '1'; -- D3,D2,D1,D0
```

```

10 with S select F <=
11     D(0) when "00", D(1) when "01",
12     D(2) when "10", D(3) when "11";
13 end rtl;

```

考试聚焦

这是组合逻辑的必考题！两种考法：(1) 2^n 选 1 MUX 实现 n 变量函数 $\rightarrow D_k$ 直接接 0/1 (2) 2^{n-1} 选 1 MUX 实现 n 变量函数 $\rightarrow$ 需要降维，剩余变量或其反接到 D 端

4.2 变式 2: 双 4 选 1 MUX 构建 8 选 1 MUX

完整教学

【结构】 两片 4 选 1 MUX 处理低 2 位地址。一片 2 选 1 MUX 用最高位地址 S_2 选择输出。总 MUX 数: 3 片 (2×4 选 1 + 1×2 选 1)。

【VHDL——结构级】

```

1  -- 片0: S1S0 选 D0-D3
2  mux_low: entity work.mux4to1 port map(D=>D(3 downto 0), S=>S(1 downto 0)
3      , Y=>F_low);
4  -- 片1: S1S0 选 D4-D7
5  mux_high: entity work.mux4to1 port map(D=>D(7 downto 4), S=>S(1 downto
6      0), Y=>F_high);
7  -- 顶层 2 选 1
8  F <= F_low when S(2)='0' else F_high;

```

4.3 变式 3: 带使能端的 MUX

完整教学

【功能】 $E=1$ 正常选择; $E=0$ 输出高阻态 'Z' (三态输出) 或固定 '0'。

【VHDL——三态输出】

```

1  F <= D(to_integer(unsigned(S))) when E='1' else 'Z';

```

【应用】多个 MUX 输出可共用一根总线（总线仲裁）。

4.4 变式 4: MUX 扩展——16 选 1

完整教学

【树形结构】4 片 4 选 1 MUX(第一层)+1 片 4 选 1 MUX(第二层)=16 选 1。总 MUX 数: 5 片。

【选择位分配】S1S0→ 第一层 (4 片), S3S2→ 第二层 (1 片)。

第五章 4 位比较器——4 大变式详解

5.1 变式 1：8 位比较器（两片 4 位级联）

完整教学

【级联方式】高位 4 位 → 主比较器。低位 4 位 → 从比较器，输出接主比较器的级联输入端。

【级联逻辑】

- 主比较器比较 A[7:4] 和 B[7:4]
- 如果高位相等，则看级联输入（低位比较器的结果）
- 低位比较器比较 A[3:0] 和 B[3:0]，输出接到主比较器的 I_A>B, I_A=B, I_A<B

【VHDL——结构级】

```
1  -- 低位比较器
2  cmp_lo: entity work.comparator_4bit
3      port map(A=>A(3 downto 0), B=>B(3 downto 0),
4              I_gt=>'0', I_eq=>'1', I_lt=>'0', -- 最低位：假设之前相等
5              A_gt=>gt_lo, A_eq=>eq_lo, A_lt=>lt_lo);
6  -- 高位比较器
7  cmp_hi: entity work.comparator_4bit
8      port map(A=>A(7 downto 4), B=>B(7 downto 4),
9              I_gt=>gt_lo, I_eq=>eq_lo, I_lt=>lt_lo, -- 级联！
10             A_gt=>A_gt, A_eq=>A_eq, A_lt=>A_lt);
```

【行为级——直接用运算符（更简洁）】

```
1  A_gt <= '1' when A > B else '0';
2  A_eq <= '1' when A = B else '0';
3  A_lt <= '1' when A < B else '0';
```

综合器自动处理任意位宽的比较！

5.2 变式 2：只输出“相等”的比较器

完整教学

【简化】只需要 $A = B$ 信号。每一位都相等 $\rightarrow$ XNOR $\rightarrow$ 全部 AND。

【VHDL】

```

1 A_eq <= '1' when A = B else '0';
2 -- 或门级：
3 A_eq <= (A(3) xnor B(3)) and (A(2) xnor B(2)) and
4         (A(1) xnor B(1)) and (A(0) xnor B(0));

```

【应用】地址译码、标签匹配、Cache Tag 比较。

5.3 变式 3：比较器 +MUX 实现取最大值

完整教学

【电路】比较器判断 $A > B \rightarrow$ 输出作为 MUX 选择信号。 $A > B$ 时 MUX 选 A，否则选 B。

【VHDL】

```

1 max_val <= A when A > B else B;
2 min_val <= A when A < B else B;

```

【综合结果】比较器 +MUX $\rightarrow$ 单行代码综合成两者组合。

5.4 变式 4：带使能的比较器

完整教学

【功能】enable=1 时比较器工作；enable=0 时所有输出 =0。

【VHDL】

```

1 process(A, B, en)
2 begin
3     if en = '0' then
4         A_gt <= '0'; A_eq <= '0'; A_lt <= '0';
5     else
6         if A > B then A_gt <= '1'; A_eq <= '0'; A_lt <= '0';
7         elsif A = B then A_gt <= '0'; A_eq <= '1'; A_lt <= '0';
8         else A_gt <= '0'; A_eq <= '0'; A_lt <= '1';

```

```
9      end if;  
10     end if;  
11 end process;
```

第六章 4 位加法器——5 大变式详解

6.1 变式 1: 8 位/16 位加法器（级联）

完整教学

【方案】两片 4 位加法器级联。低位 C_{out} → 高位 C_{in} 。

【VHDL——结构级】

```
1 add_lo: entity work.adder_4bit port map(A=>A(3 downto 0), B=>B(3 downto
2 0),
3     Cin=>'0', Sum=>Sum(3 downto 0), Cout=>carry);
4 add_hi: entity work.adder_4bit port map(A=>A(7 downto 4), B=>B(7 downto
5 4),
6     Cin=>carry, Sum=>Sum(7 downto 4), Cout=>Cout);
```

【行为级——推荐】

```
1 signal result : std_logic_vector(8 downto 0);
2 result <= ('0' & A) + ('0' & B);
3 Sum <= result(7 downto 0); Cout <= result(8);
```

综合器自动优化任意位宽加法。

6.2 变式 2: 减法器（用加法器实现 A-B）

完整教学

【补码原理】 $A - B = A + (-B) = A + (\sim B + 1) = A + \sim B + 1$

【电路】B 取反 → 接加法器 B 输入。 $C_{in} = 1$ （实现 +1）。 C_{in} 就是补码中的“+1”。

【VHDL】

```
1 diff <= A + (not B) + '1'; -- A-B = A + (~B) + 1
2 -- 或直接用减法运算符
3 diff <= A - B;
```

考试聚焦

常考: 为什么 $C_{in} = 1$ 能实现减法? 答案: $A - B = A + (\sim B) + 1 \rightarrow +1$ 部分由 $C_{in} = 1$ 提供。

6.3 变式 3: 加法器 + 寄存器 = 累加器

完整教学

【结构】 加法器计算 $ACC + Input$ 。结果在时钟沿存入 ACC。ACC 输出反馈回加法器输入端。

【VHDL】

```

1 process(clk, rst_n)
2 begin
3   if rst_n='0' then acc <= (others=>'0');
4   elsif rising_edge(clk) then
5     acc <= acc + input_data; -- 每个时钟累加一次
6   end if;
7 end process;

```

【本质】 累加器 = 计数器的一种 (不是每次 +1, 而是 + 任意值)。

【应用】 DSP 中的乘累加 (MAC)、积分器。

6.4 变式 4: 带溢出检测的加法器

完整教学

【无符号数溢出】 $C_{out} = 1$ = 溢出 (结果超过最大值)。

【有符号数溢出】 $Overflow = C_n \oplus C_{n-1}$ (最高位进位和次高位进位异或)。

【VHDL】

```

1 signal result : std_logic_vector(4 downto 0); -- 5位
2 result <= ('0' & A) + ('0' & B);
3 Sum <= result(3 downto 0);
4 Cout <= result(4);
5 -- 有符号溢出
6 Overflow <= result(4) xor result(3); -- C4 xor C3

```

【直观判断】两个正数相加得负数 → 溢出 (正溢出) 两个负数相加得正数 → 溢出 (负溢出)

6.5 变式 5: BCD 加法器

完整教学

【问题】二进制加法后, 如果结果 >9(1001), 需要 +6(0110) 修正为 BCD 格式。

【VHDL】

```
1 signal bin_sum : std_logic_vector(4 downto 0);
2 signal bcd_sum : std_logic_vector(3 downto 0);
3 begin
4     bin_sum <= ('0' & A) + ('0' & B);
5     bcd_sum <= bin_sum(3 downto 0) when bin_sum <= 9
6         else bin_sum(3 downto 0) + "0110"; -- >9时+6修正
7     Cout <= '1' when bin_sum > 9 else '0';
```

【例子】5+7=12(1100)。1100+0110=10010→BCD: 0001 0010(12)。

第七章 级联 MUX——4 大变式详解

7.1 变式 1：用 2 选 1 MUX 构建 8 选 1 MUX

完整教学

【树形结构】3 层：第一层 4 个 → 第二层 2 个 → 第三层 1 个 = 7 个 2 选 1 MUX。

【选择位分配】

- S0 → 第一层 (4 个 MUX)：每两组数据中选一个
- S1 → 第二层 (2 个 MUX)：第一层结果中两两选择
- S2 → 第三层 (1 个 MUX)：第二层两个结果中最终选择

【通用公式】 2^n 选 1 MUX 需要 $2^n - 1$ 个 2 选 1 MUX，分 n 层。

【VHDL——generate 结构级】

```
1  -- 递归描述流水线 MUX 树
2  signal layer0 : std_logic_vector(3 downto 0);
3  signal layer1 : std_logic_vector(1 downto 0);
4  begin
5      gen0: for i in 0 to 3 generate
6          layer0(i) <= D(2*i) when S(0)='0' else D(2*i+1);
7      end generate;
8      gen1: for i in 0 to 1 generate
9          layer1(i) <= layer0(2*i) when S(1)='0' else layer0(2*i+1);
10     end generate;
11     Y <= layer1(0) when S(2)='0' else layer1(1);
12 end behavioral;
```

7.2 变式 2：用 4 选 1 MUX 构建 16 选 1 MUX

完整教学

【结构】 第一层 4 片 4 选 1 MUX，第二层 1 片 4 选 1 MUX。共 5 片。

S1S0→第一层 (4 片 4 选 1 MUX)。S3S2→第二层 (1 片 4 选 1 MUX 选择第一层输出之一)。

7.3 变式 3：不对称级联（构建 6 选 1 MUX）

完整教学

【问题】 需要 6 选 1，但 MUX 只有 2^n 选 1。用 8 选 1 实现 6 选 1（两个输入悬空或接地）。

【VHDL——4 选 1+2 选 1】

```

1  -- 第一层：4 选 1 处理 D0-D3
2  mux4: entity work.mux4to1 port map(D=>D(3 downto 0), S=>S(1 downto 0), Y
    =>mux4_out);
3  -- 顶层：2 选 1 在 mux4_out 和 D4/D5 间选择
4  Y <= mux4_out when S(2)='0' else
5      D(4) when S(1 downto 0)="00" else D(5);
6  -- 注意：S2=1 时 S1S0 只需表示 2 种状态即可

```

【简化方案】 直接用 8 选 1 MUX，D6 和 D7 接地 (固定为 0)。

7.4 变式 4：MUX 树实现逻辑函数

完整教学

【本质】 MUX= 通用逻辑实现器。MUX 树 = 多层选择 = 分而治之。

【Shannon 展开定理】 $F(x_1, x_2, \dots, x_n) = \overline{x_1} \cdot F(0, x_2, \dots, x_n) + x_1 \cdot F(1, x_2, \dots, x_n)$
这正是 2 选 1 MUX 的表达式！递归应用 = 构建 MUX 树。

【结论】 任何组合逻辑函数都可用 2 选 1 MUX 树实现——这其实就是 FPGA 中 LUT 的工作原理。

第二部分

D 触发器变式详解

第八章 D 触发器——核心变式详解

8.1 变式 1：带异步复位的 D 触发器 vs 带同步复位的 D 触发器

完整教学

【异步复位】复位不受时钟控制，rst=0 立即清零（不等时钟沿）。【同步复位】复位只在时钟沿有效，rst=0 后需等下一个时钟上升沿才清零。

【VHDL 对比】

```
1  -- 异步复位（rst在敏感列表中，且在rising_edge之前判断）
2  process(clk, rst_n)
3  begin
4      if rst_n = '0' then Q <= '0';          -- 立即响应
5      elsif rising_edge(clk) then Q <= D;
6      end if;
7  end process;
8
9  -- 同步复位（rst不在敏感列表，在rising_edge内部判断）
10 process(clk)
11 begin
12     if rising_edge(clk) then
13         if rst_n = '0' then Q <= '0';      -- 等时钟沿
14         else Q <= D;
15         end if;
16     end if;
17 end process;
```

【区别】

	异步复位	同步复位
敏感列表	clk, rst_n	clk
复位延迟	立即	等 1 个时钟周期
抗噪性	差 (毛刺可触发复位)	好 (只在沿采样)
综合结果	DFF+ 异步 CLR	DFF+ 同步 CLR

考试聚焦

必考判断：看 process 敏感列表有没有 rst_n。

- 有 → 异步复位
- 没有 → 同步复位

8.2 变式 2：带时钟使能的 D 触发器

完整教学

【功能】CE=1 时正常采样 D；CE=0 时保持 Q 不变。

【VHDL】

```
1 process(clk)
2 begin
3     if rising_edge(clk) then
4         if CE = '1' then Q <= D; end if;
5         -- CE='0': 不赋值=保持
6     end if;
7 end process;
```

【综合结果】D 触发器 + 时钟使能逻辑（MUX 反馈 Q 或采样 D）。

【应用】所有带使能的寄存器/计数器都基于这个结构。

8.3 变式 3：多位 D 触发器 = 寄存器

完整教学

【原理】N 个 D 触发器并行 → N 位寄存器。同一时钟沿采样 N 位输入。

【VHDL】

```
1 signal reg : std_logic_vector(7 downto 0);
2 process(clk) begin
3     if rising_edge(clk) then reg <= data_in; end if;
4 end process;
```

【综合】8 个 D 触发器 → 8 个 FPGA 寄存器。

【应用】流水线寄存器、数据缓冲、状态寄存器。

8.4 变式 4: D 触发器构成 2 分频器

完整教学

【原理】 $\overline{Q}$ 接回 $D \rightarrow Q(n+1) = \overline{Q(n)} \rightarrow$ 每拍翻转 $\rightarrow 2$ 分频。

【VHDL】

```
1 process(clk) begin
2     if rising_edge(clk) then Q <= not Q; end if;
3 end process;
```

【这是分频器和计数器的原子单元。】

8.5 变式 5: 建立/保持时间与最大工作频率

完整教学

【建立时间 t_{su} 】时钟沿到来前，D 输入必须稳定多久。【保持时间 t_h 】时钟沿到来后，D 输入必须保持多久。【 t_{cko} 】时钟沿到 Q 输出有效的延迟。

【最大频率】 $f_{max} = 1/(t_{cko} + t_{comb} + t_{su})$

t_{comb} = 触发器之间的组合逻辑延迟。

【考试计算】给定 $t_{cko}=2\text{ns}$, $t_{su}=1\text{ns}$, 组合逻辑延迟 $=5\text{ns}$ 。 $f_{max} = 1/(2 + 5 + 1) = 125\text{MHz}$ 。

第三部分

计数器变式详解

第九章 计数器——统一框架变式详解

9.1 变式 1：带使能的模 N 计数器

完整教学

【电路】在标准计数器上增加 en 控制。【VHDL】

```
1 if rising_edge(clk) then
2     if en = '1' then
3         if cnt = N-1 then cnt <= 0; else cnt <= cnt + 1; end if;
4     end if;
5 end if;
```

【考点】en=0 时不赋值 → 保持。这是所有可控计数器的基本范式。

9.2 变式 2：带同步加载的模 N 计数器

完整教学

【功能】load=1 时 cnt=data_in。load=0 时正常计数。【优先级】load > 计数（if-else 中 load 在前 = 优先级高）。【VHDL】

```
1 if load = '1' then cnt <= data_in;
2 elsif cnt = N-1 then cnt <= 0;
3 else cnt <= cnt + 1; end if;
```

9.3 变式 3：递增/递减双向计数器

完整教学

【VHDL】

```
1 if up = '1' then cnt <= cnt + 1; else cnt <= cnt - 1; end if;
```

【递减自然溢出】 $0000 - 1 = 1111$ (4 位无符号), 自动模 16 循环。

9.4 变式 4: 倒计时计数器 (预置 $\rightarrow 0 \rightarrow$ 预置)

完整教学

【VHDL】

```
1 if cnt = 0 then cnt <= PRE; -- 到0后重新加载
2 else cnt <= cnt - 1; end if;
```

9.5 变式 5: 级联计数器——总模值 = 乘积

完整教学

【级联方式】低位 carry $\rightarrow$ 高位 en。总 $M = M_1 \times M_2$ 。

【实例】模 8 + 模 5 级联 $\rightarrow$ 总模 40。模 60 (6 $\times$ 10) BCD 级联。

【VHDL 关键】高位以低位 carry 为时钟使能——确保高位只在低位满一轮时才 +1。

9.6 变式 6: 格雷码计数器

完整教学

【特点】相邻状态仅 1 位不同。适合 FIFO 指针、跨时钟域。【VHDL】case 枚举所有格雷码状态。【3 位格雷码序列】000-001-011-010-110-111-101-100-000。

第四部分

分频器变式详解

第十章 分频器——全部变式详解

10.1 变式 1：直接取 Q_n 实现 2^n 分频

完整教学

【原理】 Q_k 每 2^{k+1} 拍完成一次 $0 \rightarrow 1 \rightarrow 0$ 循环。

【VHDL——一行搞定】

```
1 clk_div2 <= cnt(0);    -- 2分频
2 clk_div4 <= cnt(1);    -- 4分频
3 clk_div8 <= cnt(2);    -- 8分频
```

【优点】不需要任何额外逻辑。计数器本身就是分频器。

10.2 变式 2：偶数分频（任意 M ）

完整教学

【统一模板】

```
1 if cnt = M-1 then cnt <= 0; else cnt <= cnt + 1; end if;
2 clk_out <= '1' when cnt >= M/2 else '0'; -- 50%占空比
```

10.3 变式 3：奇数分频（双沿法）

完整教学

【原理】上升沿 + 下降沿各产生一个信号 $\rightarrow$ OR 组合。得到接近 50% 占空比的奇数分频。【详见解析册题 6】

10.4 变式 4：占空比可调分频器

完整教学

【原理】 改变输出翻转的阈值。占空比 = 高拍数/M。

```
clk_out <= '1' when cnt < DUTY else '0';
```

10.5 变式 5：多级分频器（分频比相乘）

完整教学

【原理】 M1 分频 → M2 分频 = 总 M1×M2 分频。**【注意】** 后级以前级输出为时钟 → 跨时钟域风险。FPGA 中推荐用使能信号而非门控时钟。

10.6 变式 6：秒脉冲发生器

完整教学

【问题】 50MHz→1Hz。M=50,000,000。需 26 位计数器。**【VHDL】** 同偶数分频模板，M=50000000。

第五部分

序列发生器变式详解

第十一章 序列发生器——全部变式详解

11.1 变式 1：任意周期序列（计数器 + 映射表）

完整教学

【通用方案】 模 L 计数器 + 状态 → 输出映射表。**【例题】** 产生序列 10110(长度 5)。模 5 计数器 +case 映射：

```
1 case cnt is
2   when "000" => seq <= '1';
3   when "001" => seq <= '0';
4   when "010" => seq <= '1';
5   when "011" => seq <= '1';
6   when "100" => seq <= '0';
7 end case;
```

【适用范围】 任意序列。长度 L → 模 L 计数器。

11.2 变式 2：状态机型序列发生器

完整教学

【方案】 每个状态输出一个序列值。状态转移 = 序列顺序。**【优点】** 状态名有意义（如”S_1”, ”S_0”, ”S_1”, ”S_1”, ”S_0”）。但状态多时代码冗长。**【比较】** 推荐用计数器 +case——代码更简洁。

11.3 变式 3：移位寄存器 +LFSR 发生器

完整教学

【LFSR】 线性反馈移位寄存器。反馈 = 选择位的 XOR。**【应用】** 伪随机序列 (PRBS)、

CRC、扰码。【4 位 LFSR(X^4+X^3+1)】反馈 = $Q_3 \text{ xor } Q_2$ 。产生 15 个伪随机状态。【注意】全 0 状态非法——LFSR 会锁死在 0000。必须初始化为非 0 值。

11.4 变式 4: ROM/查找表型序列发生器

完整教学

【方案】预存整个序列在 ROM 中。计数器 = 地址。【优点】任意序列、可重配置。适合长序列。【VHDL】

```
1 type rom_type is array (0 to 15) of std_logic_vector(3 downto 0);  
2 constant SEQ_ROM : rom_type := ("1", "0", "1", "1", "0", ...);  
3 seq <= SEQ_ROM(to_integer(unsigned(cnt)));
```

第六部分

移位寄存器变式详解

第十二章 移位寄存器——全部变式详解

12.1 变式 1：串并转换（SIPO）

完整教学

【功能】 串行输入 → 4 拍后并行读取。UART/SPI 接收器的基础。**【VHDL】** `reg <= si & reg(N-1 downto 1); po <= reg;`

12.2 变式 2：并串转换（PISO）

完整教学

【功能】 load 并行数据 → 逐拍串行输出。UART/SPI 发送器的基础。**【VHDL】**

```
1 if load='1' then reg <= data_in;  
2 else reg <= '0' & reg(N-1 downto 1); end if;  
3 so <= reg(0);
```

12.3 变式 3：双向移位

完整教学

【VHDL】

```
1 if dir='1' then reg <= si_r & reg(N-1 downto 1);  
2 else reg <= reg(N-2 downto 0) & si_l; end if;
```

12.4 变式 4：循环移位

完整教学

【功能】 数据不丢失——最低位移回最高位。**【VHDL】** `reg <= reg(0) & reg(N-1 downto 1);`（右循环）**【本质】** 这就是环形计数器的基础！

12.5 变式 5：算术移位（保持符号位）

完整教学

【逻辑右移】 高位补 0 $\rightarrow$ `'0' & reg(N-1 downto 1)` **【算术右移】** 高位补符号位 $\rightarrow$ `reg(N-1) & reg(N-1 downto 1)` **【应用】** 有符号数 $\div 2$ 操作。

第七部分

环形计数器变式详解

第十三章 环形计数器——全部变式详解

13.1 变式 1: N 位环形计数器

完整教学

【通用模板】

```
1 process(clk, rst_n)
2 begin
3   if rst_n='0' then reg <= (N-1=>'1', others=>'0'); -- One-hot 初值
4   elsif rising_edge(clk) then
5     reg <= reg(0) & reg(N-1 downto 1); -- 右移+反馈
6   end if;
7 end process;
```

【结论】 N 个 DFF→N 个状态。所有状态中恰好一个 1（One-hot 编码）。

13.2 变式 2: 约翰逊计数器（2N 个状态）

完整教学

【反馈】 `reg <= (not reg(0)) & reg(N-1 downto 1);` **【4 位约翰逊状态序列】**
0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 → 0000 **【结论】**
N 个 DFF→2N 个状态（比环形计数器多一倍）。

13.3 变式 3: 自启动环形计数器

完整教学

【问题】 干扰可能使环形计数器进入非法状态。**【方案】** 检测非法状态（如全 0 或多个 1），强制恢复到合法状态 (1000)。**【VHDL】** 在移位前检查: `if count_ones(reg)`

```
/= 1 then reg <= "1000";
```

13.4 变式 4：用环形计数器实现时序控制器

完整教学

【应用】多周期操作器的步骤控制。**【5 步控制器】**5 位环形计数器 → 每拍恰好一个步骤信号为 1 → 控制对应步骤。**【优点】**无需额外的译码逻辑——直接读 Q 值。

第八部分

序列检测器变式详解

第十四章 序列检测器——全部变式详解

14.1 变式 1：检测 1011（重叠 vs 非重叠）

完整教学

【核心区别】S101+ 输入 1→检测到！

- 非重叠 → 回 S0（重新开始）
- 重叠 → 去 S1（最后的”1”作为新序列的第一个”1”）

【考试判断】看检测到后的转移目标。S0= 非重叠，S1(或其他非 S0 状态)= 重叠。

14.2 变式 2：检测 1101

完整教学

【状态】S0→S1→S11→S110→(输入 1=检测到) 【重叠处理】S110+1→检测到 → 去 S11（”1101”中后两位”11”匹配”11”前缀）【关键】S11+1→S11（”111”中后两个”11”为有效前缀）。

14.3 变式 3：检测 1111（连续 4 个 1）

完整教学

【状态】S0→S1(1 个 1)→S11(2)→S111(3)→(第 4 个 1→检测) 【特点】任意时刻输入 0→回 S0（连续中断）。【重叠】S111+1→S111（”1111”后 3 个”111”为有效前缀）。

14.4 变式 4：检测 0101

完整教学

【状态】 $S_0 \rightarrow S_01(0) \rightarrow S_010(01) \rightarrow S_0101(\text{检测})$ **【注意】** $S_0101+0 \rightarrow S_01$ （重叠：最后的”0”作为新序列的第一个”0”）。

14.5 变式 5：用移位寄存器 + 比较器替代状态机

完整教学

【方案】 移位寄存器存最近 N 位 $\rightarrow$ 与目标序列比较。 **【VHDL——两行搞定】**

```
1 reg <= din & reg(3 downto 1);
2 detected <= '1' when reg = "1011" else '0';
```

【优点】 代码极简。不需推导状态转移。 **【注意】** 自动实现重叠检测。 **【考试适用】** 除非题目明确要求”用状态机设计”，否则移位寄存器方案是最佳选择。

第九部分

VHDL 综合变式详解

第十五章 VHDL 综合——全部变式详解

15.1 变式 1: if-else 的综合结果

完整教学

【综合规则】

- if-else 级联 → 优先级 **MUX 链**。上层 if 优先级最高。
- 带 clk 的 if → 产生 **寄存器 (D 触发器)**。
- 不带 clk 的 if → 产生 **组合逻辑**。
- 不完整的 if(缺少 else) → 可能产生 **锁存器 (Latch)**——通常应避免。

【考试判断】

```
1  -- 组合逻辑MUX
2  if sel='1' then y <= a; else y <= b; end if;
3
4  -- 时序逻辑（寄存器）
5  if rising_edge(clk) then
6      if en='1' then q <= d; end if;  -- 带使能的D触发器
7  end if;
```

15.2 变式 2: case 的综合结果

完整教学

【综合规则】

- case → **并行 MUX 或译码器**。所有分支并行判断，无优先级。
- case 不完备且无 when others → 可能产生锁存器。

- case 完备 (有 when others)→ 纯组合逻辑或状态机。

【if-else vs case 硬件对比】

	if-else	case
综合结果	优先级编码器/级联 MUX	并行 MUX/译码器
优先级	有 (从上到下递减)	无 (并行)
速度	慢 (级联延迟)	快 (并行)
面积	小 (级联结构)	大 (并行结构)

15.3 变式 3: signal vs variable

完整教学

【**signal(<=)**】赋值在 process 结束时更新。适合跨进程通信和寄存器。【**variable(:=)**】赋值立即更新。适合 process 内部中间计算。

【综合结果】

- signal 在时序 process 中 → D 触发器
- signal 在组合 process 中 → 连线
- variable → 通常不产生额外硬件 (被综合器优化为连线)
- 但 variable 在读取前先赋值 → 可能综合为寄存器

【**考试判断**】看赋值符号: <= (信号) vs := (变量)。

15.4 变式 4: process 的综合结果

完整教学

【**process 本身不产生硬件**】硬件由 process 内部代码决定。

process 写法	综合结果
process(clk)+rising_edge	D 触发器/寄存器
process(all_inputs)+ 无 clk	组合逻辑
process(clk,rst)+ 异步 rst	D 触发器 + 异步复位
不完整敏感列表	可能锁存器 (应避免)

15.5 变式 5：常见 VHDL 模式 → 硬件对照表

完整教学

VHDL 模式	硬件
if rising_edge(clk) then Q<=D;	1 个 D 触发器
y <= a when sel='1' else b;	2 选 1 MUX
with sel select y<=...	并行 MUX
case state is when...	状态机 + 译码器
reg <= si & reg(N-1 downto 1);	移位寄存器
cnt <= cnt + 1;	加法器 + 寄存器
if cnt=N-1 then cnt<=0;	比较器 (清零条件)
for i in 0 to 7 generate	8 个相同的硬件副本

考试聚焦

VHDL 综合考点总结：

- 有时钟边沿 = 有触发器
- if-else= 优先级，case= 并行
- signal<= 延迟更新，variable:= 立即更新
- process 只是容器，内部代码决定硬件
- for-generate= 复制硬件 N 份